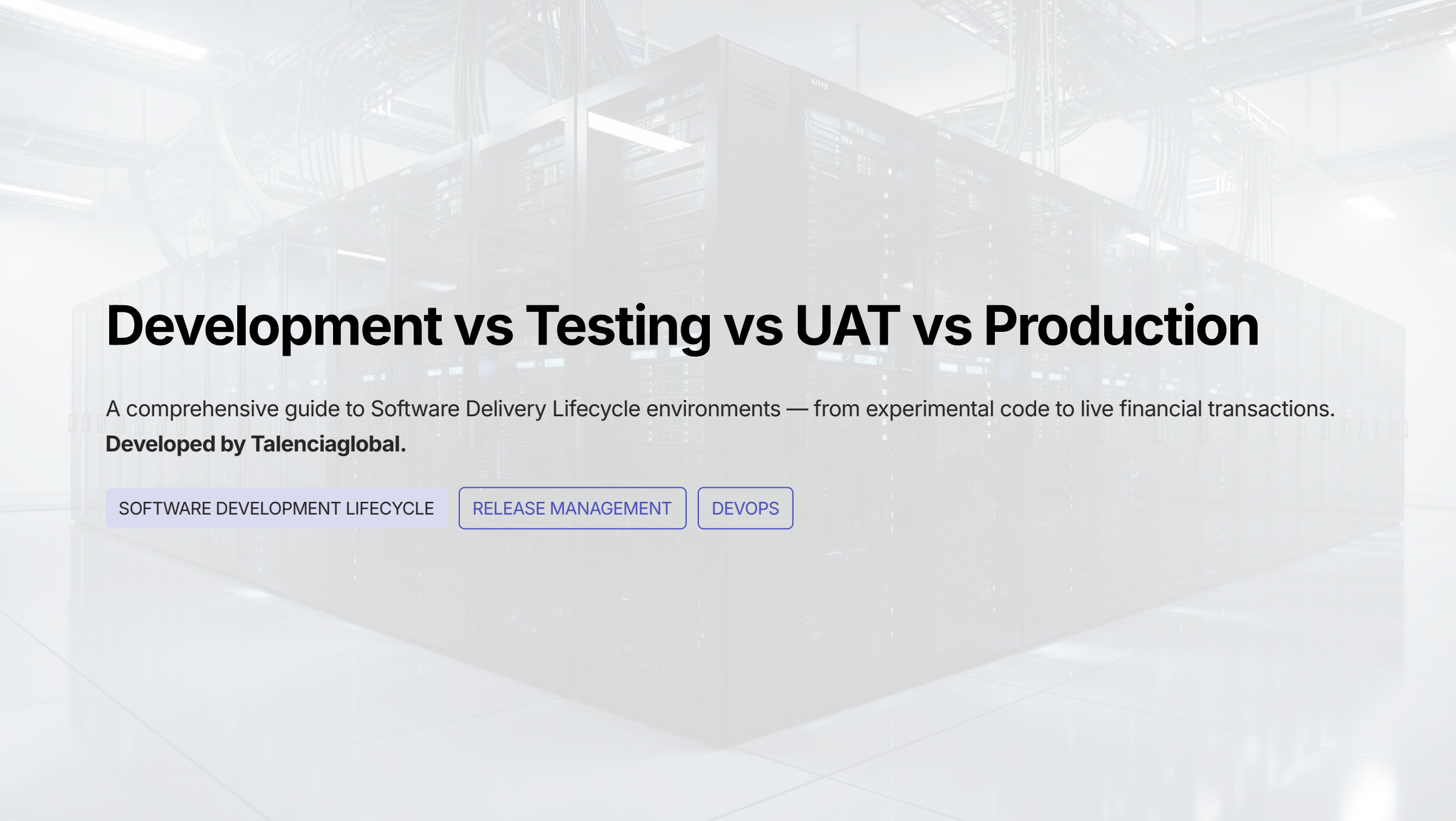


Development vs Testing vs UAT vs Production

A comprehensive guide to Software Delivery Lifecycle environments — from experimental code to live financial transactions.

Developed by Talenciaglobal.

SOFTWARE DEVELOPMENT LIFECYCLE | RELEASE MANAGEMENT | DEVOPS

Topic Overview

Category

Software Development Lifecycle (SDLC) / DevOps — Release Management & Quality Assurance

Difficulty Level

Beginner to Intermediate — requires only a basic understanding of software development concepts

Industry Relevance

Critical in FinTech — RBI mandates separate UAT environments with production-like data masking

Topic Type

Process & Workflow Framework — governing how software moves from idea to live customer impact

The recommended learning path follows the natural delivery sequence: **Development → Unit Testing → Integration Testing → UAT → Staging → Production → Post-deployment Validation.**

The Restaurant Analogy: Understanding Environments Intuitively

Software environments can be understood through a powerful analogy — think of them as stages in a restaurant chain's kitchen operation.

DEV — The Test Kitchen

Chefs experiment with new recipes. Spicy sauces, new ingredients, radical ideas. Nobody eats this food. It is messy, experimental, and changes every hour. Failure is expected and encouraged.

TEST/QA — The Quality Control Lab

A small team tastes the dish, checks temperature, and ensures it will not harm customers. They find bugs like "too salty" or "undercooked" before any real patron is served.

UAT — The VIP Preview Night

Real customers — not chefs — are invited to taste before the public launch. They verify the dish solves their actual hunger. "Yes, this works for my dietary needs."

PROD — The Live Restaurant

10,000 paying customers dine here daily. A wrong dish triggers complaints, refunds, and reputation damage. Zero tolerance for errors at this stage.

FinTech Analogy: A new UPI payment feature must traverse DEV (engineer codes it), TEST (automated checks), UAT (RBI auditors and bank partners validate), before PRODUCTION (10 million users send real money).

Formal Definition: The Four Environments

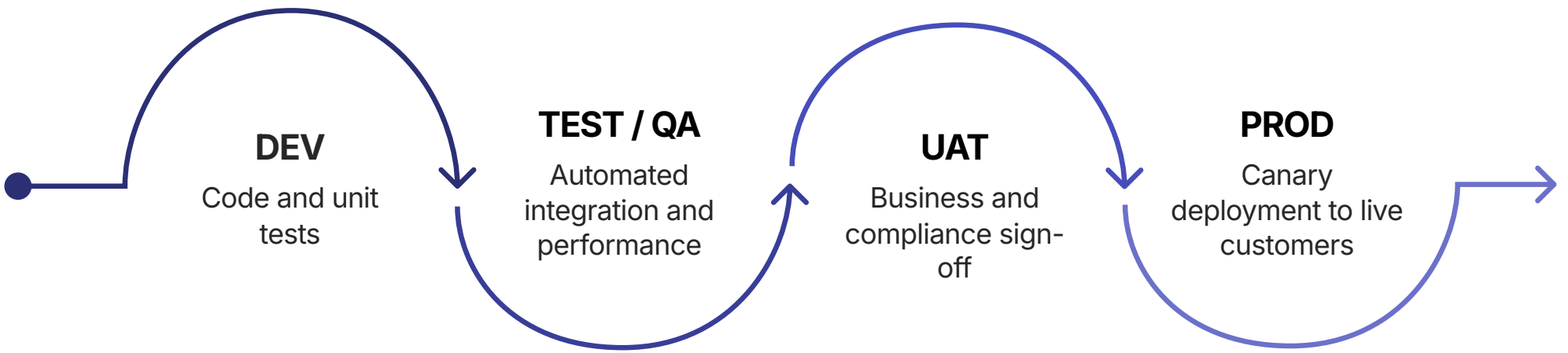
Each environment represents a distinct stage in the software delivery lifecycle, with specific purposes, access controls, data characteristics, and quality gates.

Environment	Purpose	Users	Data	Stability
Development (DEV)	Build and unit test features	Developers only	Synthetic / mocked	Unstable (changes hourly)
Testing (TEST/QA)	Automated integration & regression testing	QA engineers, automated suites	Anonymized test data	Semi-stable (changes daily)
UAT	Business validation & compliance sign-off	Business users, clients, auditors	Production-like (masked)	Stable (changes weekly)
Production (PROD)	Live service for real customers	End users, customers	Real sensitive data	Immutable (change-controlled)

Industry statistic: Organizations with well-defined environment separation report **73% fewer production incidents** and **60% faster mean time to recovery (MTTR)** compared to those without structured pipelines. (Source: DORA State of DevOps Report)

The Enterprise FinTech Deployment Pipeline

The following pipeline illustrates a complete financial services delivery workflow, using a UPI Payment 2FA Enhancement as the reference feature throughout this guide.



Each stage acts as a quality gate — code cannot advance to the next environment without satisfying defined promotion criteria. This structured progression is the foundation of reliable, compliant software delivery in regulated industries.

Stage 1: The Development Environment

The Development environment is where software is born. It is the most fluid, experimental, and change-tolerant stage in the pipeline — designed for speed of iteration, not stability.

Core Characteristics

- Primary Goal: Write, test, and iterate on features rapidly
- Users: Software engineers (individual contributors)
- Setup: Local laptop (Docker Compose) or shared Dev K8s namespace
- Data: Synthetic — "John Doe", "1234567890" (Java Faker library)
- Deployment Frequency: Multiple times per hour (auto-deploy on commit)
- Uptime SLA: None — environment can be down without consequence
- Compliance: None — no real data, no audit required

CI/CD Configuration

- Feature branch → Jenkins / GitHub Actions
- Compile (30 seconds)
- Unit tests — JUnit, Mockito (1 minute)
- SonarQube static analysis (2 minutes)
- Build Docker image (1 minute)
- Deploy to Dev Kubernetes namespace (30 seconds)

Quality Gate

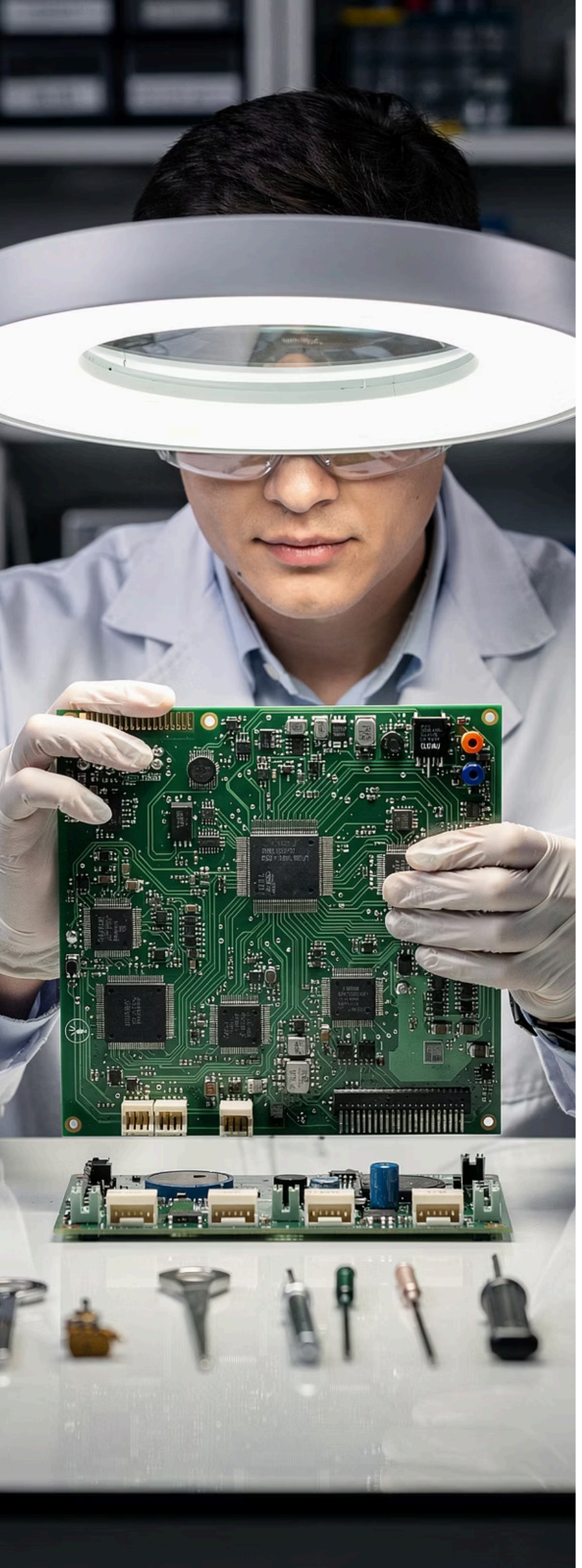
Unit test coverage ≥ 90%, zero critical SonarQube issues, no hardcoded secrets in code.

DEV Environment: Developer Workflow in Practice

The following illustrates a real developer workflow implementing the UPI 2FA feature — from local coding to CI pipeline execution.

```
# Developer implements 2FA feature
git checkout -b feature/upi-2fa

# Write code (Spring Boot + Twilio for SMS OTP)
# Run unit tests
mvn test                # 500 tests, 10 seconds

# Run local integration (Testcontainers - PostgreSQL + Redis)
mvn integration-test    # 2 minutes

# Commit and push
git commit -m "feat: add 2FA for UPI payments > ₹2000"
git push origin feature/upi-2fa

# CI pipeline triggers automatically:
# - Compile          (30s)
# - Unit tests       (1 min)
# - SonarQube analysis  (2 min)
# - Build Docker image  (1 min)
# - Deploy to Dev K8s   (30s)
```

Best Practice: Configuration file `application-dev.yml` uses H2 in-memory database, debug logging, and mock Twilio credentials. Production credentials are strictly forbidden in the DEV environment — a PCI-DSS requirement that prevents accidental data exposure.



Stage 2: The Testing Environment (QA)

The Testing environment is where automated rigor replaces manual experimentation. Every integration point, performance boundary, and security surface is systematically validated before code advances further.

Integration Tests

API tests via Postman/Newman (100 cases), contract tests via Pact (50 contracts), end-to-end via Cypress/Selenium (200 scenarios)

Performance Tests

Gatling / JMeter simulating 10,000 concurrent users. SLA: p99 latency < 200ms, error rate < 0.1% at 2× peak load

Security Scans

SAST, DAST (OWASP ZAP), Snyk dependency scanning for critical CVEs, SQL injection and XSS validation

Chaos Tests

Gremlin / manual pod deletion — kill Redis master, verify circuit breaker opens, confirm graceful degradation

QA Environment: Key Specifications

Aspect	Detail
Primary Goal	Validate integration, performance, security, regression
Users	QA engineers, automation engineers, security team
Setup	Dedicated K8s cluster (prod config, scaled down)
Data	Anonymized production copy (masked PAN: XXXX-XXXX-XXXX-1234)
Deploy Frequency	Multiple times per day (on main branch merge)
Uptime SLA	99% (business hours only)
Rollback	Automated — quality gate failure reverts to last good build
Compliance	Pre-audit — verify audit trails function before production
Cost	Moderate — spot instances, scale to zero at night

Quality Gate Criteria

- All integration tests pass (500+ cases)
- Performance SLA met: p99 < 200ms
- No critical CVEs in dependency scan
- Code coverage > 80% (unit + integration)
- Chaos tests pass — circuit breakers verified
- No open P1 or P2 defects
- Data masking verified — no real PAN in TEST

Industry benchmark: Elite DevOps teams run **500+ automated tests** in QA, achieving deployment frequencies of multiple times per day with a change failure rate below **5%**.

QA Pipeline: Automated Test Execution

The following pipeline configuration illustrates the full automated test suite executed upon every merge to the main branch in a FinTech QA environment.

```
stages:
 - deploy-qa:
 kubectl apply -f qa/
 wait_for_ready: 5 minutes

 - integration-tests:
 newman run upi-2fa-collection.json
 # 100 API tests, 10 minutes

 - regression-tests:
 cypress run --spec "cypress/e2e/**/*"
 # 200 E2E tests, 30 minutes

 - contract-tests:
 pact-verifier --provider-base-url=http://payment-service-qa
 # 50 contracts, 5 minutes

 - performance-tests:
 gatling -s Upi2FASimulation
 # 10,000 concurrent users, 20 minutes
 # Assert: p99 < 200ms, error rate < 0.1%

 - security-scans:
 snyk test --severity high # CVE dependency check
 zap-full-scan.py -t http://payment-service-qa # DAST, 30 min

 - chaos-tests:
 kubectl delete pod redis-master-0
 verify: payment service returns fallback (5xx gracefully)

 - quality-gate:
 fail if: any test fails | SLA breached | critical CVE | coverage < 80%
```

Stage 3: UAT — User Acceptance Testing

UAT is the critical bridge between technical correctness and business validity. It is where the question shifts from *"Does it work?"* to *"Does it solve the right problem — compliantly?"*

1

Deploy to UAT Cluster

Release candidate promoted from QA — automated deployment to production-like environment (100 pods, 10% of prod scale)

2

Business Validation

Product owners and business analysts verify all user stories and acceptance criteria against real business scenarios

3

Compliance Verification

RBI auditors, PCI-DSS officers, and legal teams validate regulatory requirements, audit trails, and data localization

4

Go / No-Go Sign-off

Formal approval meeting — all stakeholders sign off before Change Advisory Board (CAB) submission for production

UAT Environment: Specifications & Participants

Environment Specifications

- Setup: Production-like K8s cluster (10% of prod scale — 100 pods)
- Data: Production copy with masked PII — realistic volume of 10M rows
- Config: application-uat.yml — production config with test external APIs
- Deploy Frequency: Weekly (release candidates only)
- Uptime SLA: 99.5% (approval windows only)
- Security: Full production security controls (except real encryption keys)
- Rollback: Manual — revert to previous release candidate if sign-off fails

Participants & Roles

- **Product Owners:** Does this solve the customer problem?
- **Business Analysts:** Does this match documented requirements?
- **Compliance Officers:** RBI, PCI-DSS — is the audit trail complete?
- **RBI Auditors:** Regulatory mandate validation (e.g., 2FA for UPI > ₹2000)
- **Bank Partners:** HDFC, ICICI — settlement and reconciliation checks
- **Beta Customers:** 100 selected users for real-world scenario validation

⚠ RBI Circular 2024-03 mandates that UAT environments must use production-like data masking and that compliance officers must formally sign off before any payment feature reaches production.

UAT Sign-off Checklist: UPI 2FA Feature

The following represents a formal UAT sign-off checklist for the UPI 2FA enhancement — a real-world example of multi-stakeholder compliance validation.

Business Validation — Product Owner

- [x] ~~Customer can initiate UPI payment > ₹2000~~
- [x] ~~SMS OTP received within 5 seconds~~
- [x] ~~Payment completes after OTP entry~~
- [x] ~~Transaction appears in history~~
- [x] ~~Email receipt sent to customer~~
- [ ] Sign-off: Rajesh Sharma (Product Owner)

Compliance Validation — RBI Officer

- [x] ~~Audit log captures: user_id, amount, timestamp, OTP result~~
- [x] ~~2FA enforced for all transactions > ₹2000 (RBI circular 2024-03)~~
- [x] ~~Data localization: OTP logs stored in India region only~~
- [x] ~~Retention policy: 7 years in immutable storage~~
- [ ] Sign-off: RBI Audit Team

Bank Partner Validation — HDFC Representative

- [x] ~~Settlement file includes 2FA reference ID~~
- [x] ~~Reconciliation matches (internal vs NPCI records)~~
- [x] ~~Chargeback handling tested (disputed 2FA transaction)~~
- [ ] Sign-off: HDFC Integration Team

Customer Validation — 100 Beta Users

- [x] ~~95% success rate for first-time use (survey)~~
- [x] ~~Average completion time: 15 seconds~~
- [x] ~~Support tickets: 2 (forgotten OTP — acceptable)~~
- [ ] Sign-off: Customer Experience Team

Go / No-Go Decision

- [x] ~~All sign-offs obtained~~
- [x] ~~No open P1/P2 bugs~~
- [x] ~~Performance SLA validated (p99: 180ms)~~
- [x] ~~Rollback plan documented~~

DECISION: GO FOR PRODUCTION DEPLOYMENT

Stage 4: The Production Environment

Production is where software meets reality. It is the most consequential, most protected, and most scrutinized environment in the entire pipeline — where real customers transact real money under real regulatory oversight.

99.99%

Uptime SLA

Only 4 minutes of allowable downtime per month — equivalent to 52 minutes per year

10M+

Active Users

Real customers transacting real money — every second of downtime has measurable financial impact

7yr

Audit Retention

All production transactions logged immutably for 7 years per RBI and PCI-DSS mandates

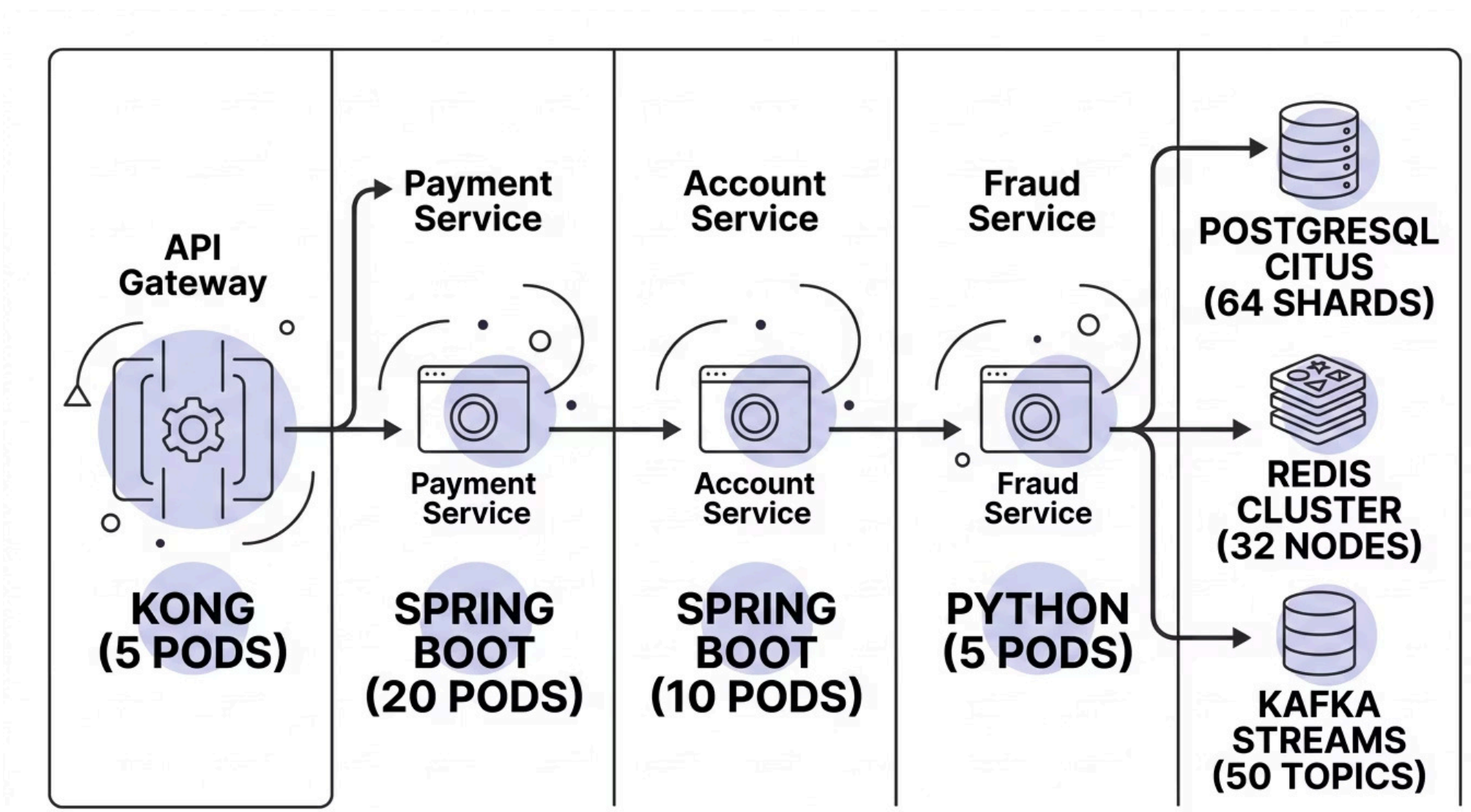
24/7

SRE Coverage

Site Reliability Engineers on-call around the clock with PagerDuty escalation protocols

Production Architecture: Enterprise FinTech Stack

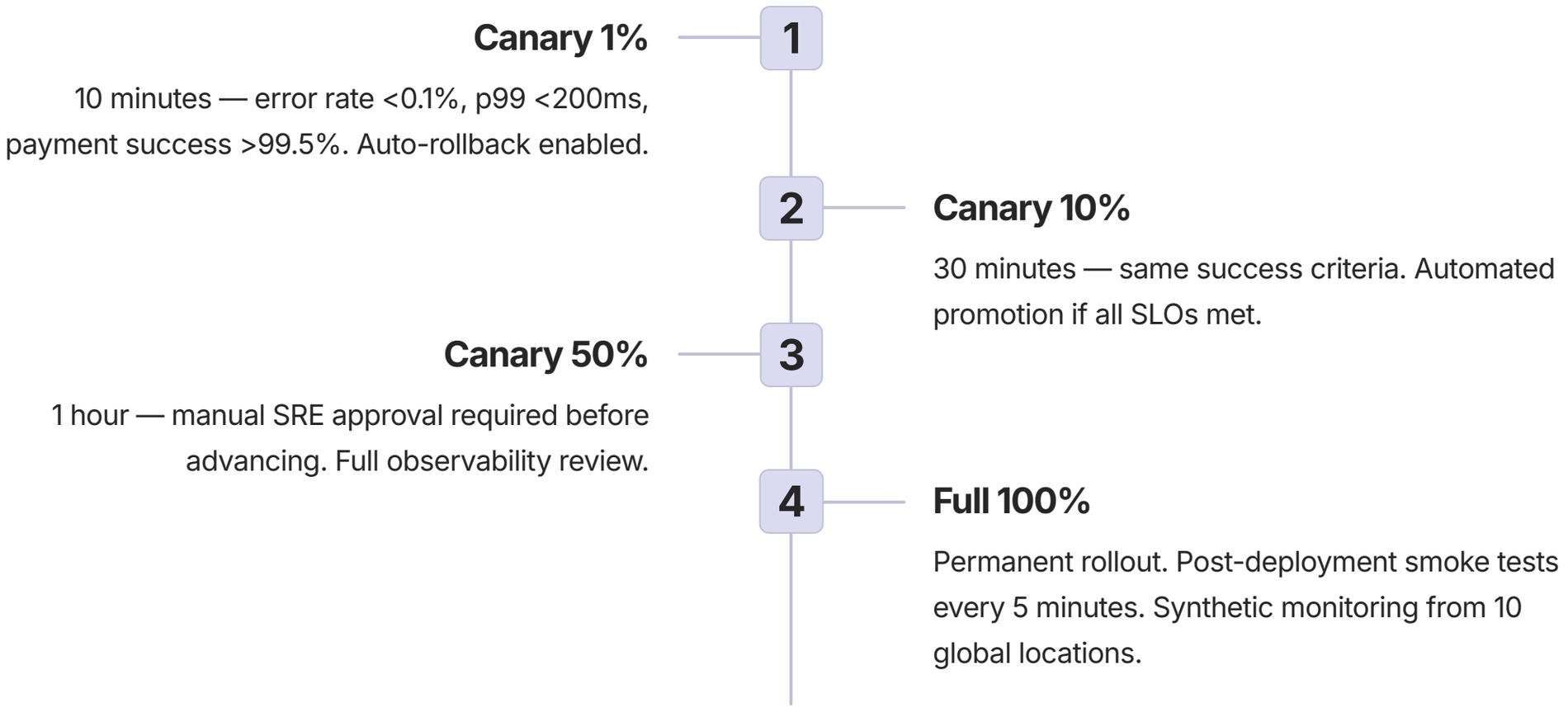
The production environment for a major UPI payment platform operates at a scale that demands multi-AZ, multi-region Kubernetes orchestration with full zero-trust security.



This architecture supports canary deployments, blue-green traffic shifting, and automated rollback — ensuring that even a flawed release affects only a fraction of users before being caught and reverted.

Production Canary Deployment Strategy

No change reaches 100% of production traffic immediately. The canary strategy progressively exposes new code to increasing traffic percentages, with automated rollback at each stage.



Industry Impact: Organizations using canary deployments reduce the blast radius of production incidents by up to **98%** — a 45-minute full outage becomes a 2-minute partial impact affecting only 1% of users.

Production Deployment Configuration

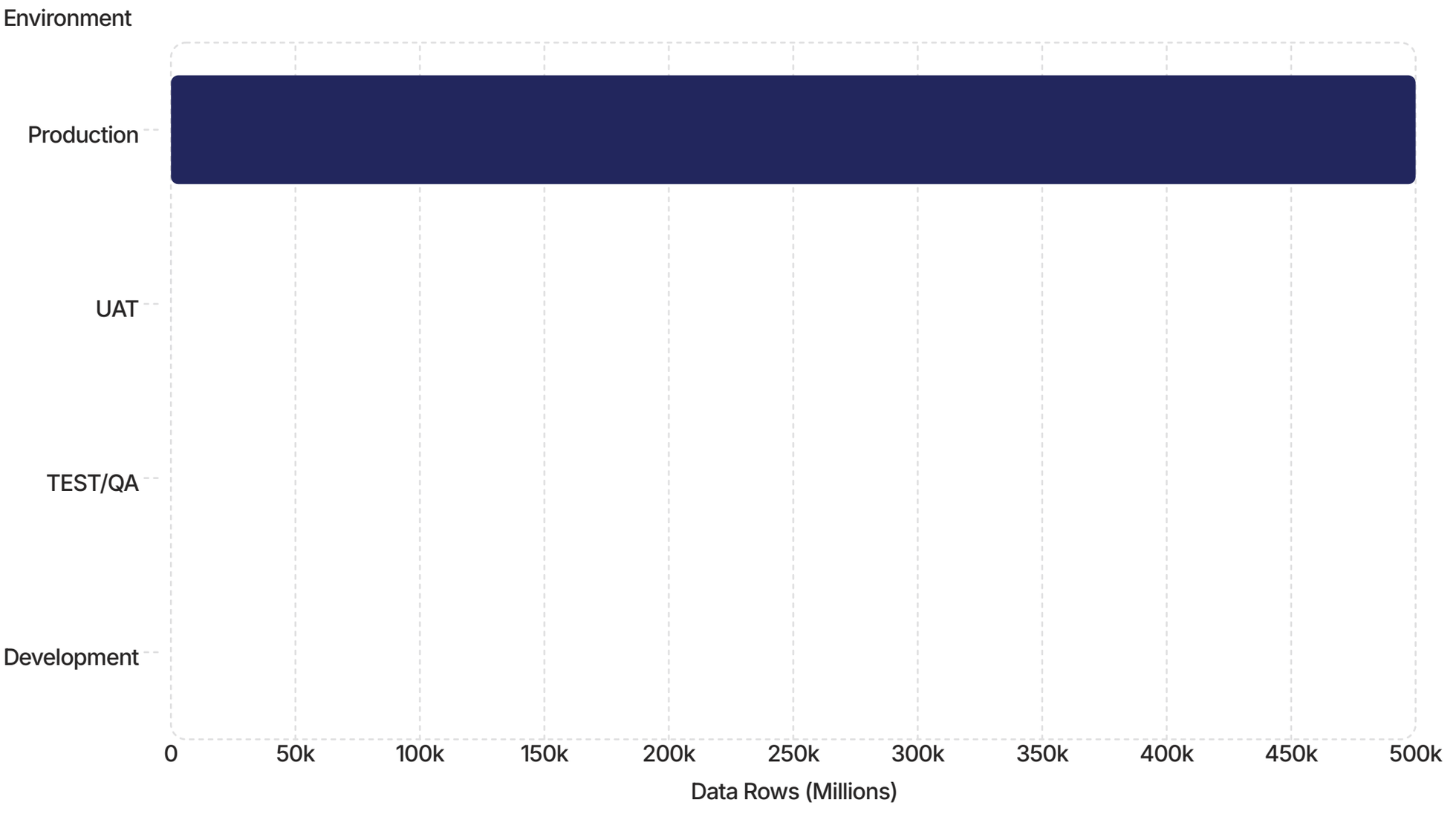
```
deployment:
 name: upi-2fa-feature
 strategy: canary
 stages:
 - name: canary-1%
 duration: 10 minutes
 traffic_weight: 1%
 success_criteria:
 - error_rate < 0.1%
 - p99_latency < 200ms
 - payment_success_rate > 99.5%
 auto_rollback: true

 - name: canary-50%
 duration: 1 hour
 traffic_weight: 50%
 manual_approval_required: true # SRE on-call must approve
 auto_rollback: true

 - name: full-100%
 traffic_weight: 100%
 post_deployment_tests:
 - smoke_tests: every 5 minutes (synthetic)
 - e2e_tests: hourly (from global locations)

post_deployment:
 - name: slo-monitoring
 continuous: true
 slos:
 - availability: 99.99% over 30 days
 - latency: p99 < 200ms over 5 minutes
 - error_rate: < 0.1% over 1 minute
 alerting:
 - if error_budget_consumed > 50%: page on-call (P2)
 - if error_budget_consumed > 100%: page incident commander (P1)
```

Comprehensive Environment Comparison

The following table provides a definitive side-by-side comparison across all critical dimensions — the authoritative reference for environment governance decisions.

Dimension	DEV	TEST/QA	UAT	PROD
Purpose	Build features	Find bugs	Validate business	Serve customers
Users	Developers	QA engineers	Business + clients	Real customers (millions)
Data	Synthetic (Faker)	Anonymized prod copy	Masked prod (10% vol)	Real sensitive data
Data Volume	1,000 rows	1M rows	10M rows	500B+ rows
Deploy Frequency	Multiple/hour	Multiple/day	Weekly	Daily (low-risk)
Change Lead Time	Minutes	Hours	Days	Weeks (CAB)
Testing	Unit only	Automated suite	Manual + business	Canary + synthetic
Uptime SLA	None	99% (biz hours)	99.5% (windows)	99.99% (24/7)
Security	None	Test credentials	Prod controls (no keys)	Zero-trust + audit
Compliance	N/A	Pre-audit	Sign-off required	Full compliance
Rollback Time	Seconds	Minutes	Hours	Seconds (auto)
Who wakes at 3AM	Nobody	Nobody	Nobody	SRE on-call

Data Volume Across Environments



The exponential difference in data volume across environments underscores why production-like testing in UAT — at 10% of production scale — is essential for validating performance characteristics before full rollout. A system that performs acceptably at 1M rows may exhibit entirely different behavior at 500 billion rows.

Case Study: PhonePe UPI 2FA Feature — 6-Week Deployment Timeline

Feature: Add SMS OTP verification for UPI payments exceeding ₹2,000 — mandated by RBI Circular 2024-03. This case study illustrates the complete journey from developer laptop to 10 million live users.

Weeks 1–2: DEV

5 engineers write Spring Boot code, Twilio SMS integration, and unit tests. 500 unit tests, 90%+ coverage achieved.

Week 3: TEST/QA

3 QA engineers run 500 integration cases, performance test at 10,000 TPS, and complete DAST security scan.

Week 4: UAT

Product Owner, RBI auditors, and 100 beta customers validate. 95% first-time success rate. Formal sign-off obtained.

Week 5: Staging

SRE team executes final pre-production validation including blue-green swap test and runbook verification.

Week 6: PROD

Canary deployment: 1% → 10% → 50% → 100% over 48 hours. Zero incidents. Feature live for 10 million users.

Environment Configuration: DEV to PROD

Configuration management is one of the most critical aspects of environment governance. The following illustrates how the same application is configured differently across all four stages.

DEV Configuration

```
# application-dev.yml
spring:
  datasource:
    url: jdbc:h2:mem:testdb  # In-memory, resets daily
  redis:
    host: localhost:6379     # Docker container
twilio:
  mock: true               # No actual SMS sent
logging:
  level: DEBUG
```

TEST/QA Configuration

```
# application-qa.yml
spring:
  datasource:
    url: jdbc:postgresql://qa-postgres:5432/payments
    hikari.maximum-pool-size: 20
  redis.cluster:
    nodes: qa-redis-1:6379,qa-redis-2:6379
twilio:
  mock: false
  test-mode: true          # Test numbers only
logging:
  level: INFO
```

UAT Configuration

```
# application-uat.yml
spring:
  datasource:
    url: jdbc:postgresql://uat-postgres:5432/payments?
ssl=true
    hikari.maximum-pool-size: 100
twilio:
  mock: false
  production-account: false  # UAT Twilio account
  from-number: "+19876543210"
audit:
  enabled: true            # Verify audit logs work
logging:
  level: WARN
```

PRODUCTION Configuration

```
# application-prod.yml
spring:
 datasource:
 url: jdbc:postgresql://prod-
postgres.citus:5432/payments?ssl=true
 hikari.maximum-pool-size: 500
 redis.cluster:
 nodes: prod-redis-1..32 # 32 nodes
twilio:
 mock: false
 production-account: true
 from-number: "+919876543210"
audit:
 enabled: true
 retention: 7-years
 immutable: true
logging:
 level: ERROR # Minimal for performance
```

Regulatory Requirements: Why Environments Are Mandated

In regulated industries, environment separation is not merely a best practice — it is a legal obligation enforced by multiple regulatory bodies with significant financial penalties for non-compliance.

Regulation	Environment Requirement
RBI (KYC/AML)	UAT must have production-like data masking — realistic but fully anonymized. Compliance officer sign-off mandatory before production deployment of any payment feature.
PCI-DSS	Production PAN (card numbers) must be encrypted at rest and in transit. DEV and TEST environments are strictly prohibited from using real PAN data — violation triggers Level 1 audit.
GDPR	UAT requires a Data Protection Impact Assessment (DPIA) before any real customer data — even masked — is used. Right to erasure must be testable in UAT.
SOX (Trading)	Separate environments are mandatory. Segregation of duties is enforced: developers cannot deploy to production. All production changes require documented approval chain.

⊗ **Financial Consequence:** A FinTech firm found using real PAN data in a development environment faces PCI-DSS Level 1 audit, potential fines exceeding **₹10 crore**, and mandatory quarterly forensic audits for a minimum of 12 months.

Risk Mitigation: The Cost of Skipping Environments

Without Separate Environments

Developer codes → deploys directly to production → bug causes double debit → ₹10 crore financial loss → RBI penalty → reputation damage → customer churn

---

This is not a hypothetical. In 2023, a major Indian payment processor suffered a ₹47 crore loss due to a double-debit bug that reached production without UAT validation.

With Separate Environments

- **DEV:** Developer finds logic error in unit test — 10 minutes to fix
- **TEST:** QA finds integration bug with Redis — 2 hours to fix
- **UAT:** RBI auditor identifies missing audit trail — 1 day to fix
- **PROD:** Canary catches regression at 1% traffic — auto-rollback in 30 seconds

Result: Bug caught before reaching customers. **Zero financial loss. Zero regulatory penalty. Zero reputation damage.**

The "Environments as Controls" Model

Each environment enforces a progressively stricter set of controls — forming a layered defense that prevents defects, security vulnerabilities, and compliance failures from reaching production.

Control	DEV	TEST	UAT	PROD
Code review required	Yes	Yes	Yes	Yes
Automated tests must pass	Unit only	All (500+)	Regression	Canary
Manual approval	No	No	Business sign-off	CAB approval
Secrets access	Mock only	Test keys	Test keys	Real Vault
Database write access	Full	Full (test only)	Restricted	Immutable (audited)
Deployment permission	Developer	CI/CD system	Release manager	SRE (break-glass)
Audit logging	No	No	Yes (test)	Yes (real, 7-year)

Common Pitfalls: What NOT to Do

The following anti-patterns represent the most frequently observed — and most costly — environment governance failures in FinTech organizations.

DEV = PROD Configuration

Developer uses production Redis credentials → accidentally flushes cache → ₹5 crore outage. **Never allow production credentials in development environments.**

No UAT Environment

Compliance bug found in production → RBI penalty of ₹10 crore. UAT is not optional in regulated industries — it is a legal requirement.

Real Data in DEV

PAN numbers leaked from a developer's laptop → PCI-DSS violation → mandatory Level 1 forensic audit → customer notification obligations.

Skipping TEST Environment

Performance bug reaches production → database connection pool exhaustion → UPI outage lasting 2 hours → ₹15 crore in failed transactions.

No Canary in Production

Bad release takes down 100% of traffic immediately → 45-minute outage → versus 2-minute partial impact with canary deployment at 1% traffic.

Manual Deployments Only

Human error in production configuration → wrong database connection string → DEV environment accidentally pointed to PROD database → data corruption risk.

Best Practices: The Gold Standard for Environment Governance

Infrastructure as Code

All environments defined identically using Terraform, parameterized by stage. Eliminates "works on my machine" — the single most common source of environment-related production incidents.

Configuration as Code

ConfigMaps and Secrets stored in Git — never hand-edited in any environment. All configuration changes are auditable, reviewable, and reversible.

Database Migrations Tested

Flyway / Liquibase migrations validated in DEV, TEST, and UAT before production. Checksums prevent unauthorized schema changes. Rollback scripts mandatory.

Automated Data Masking

Delphix or custom masking pipeline ensures no real PII reaches lower environments. Masking is automated — never a manual step that can be forgotten under deadline pressure.

Automated Promotion Gates

Jenkins / Spinnaker with automated quality gates eliminate human error in environment promotions. No manual "I think it's ready" decisions — only objective criteria.

One-Click Rollback

Kubernetes rollback and database point-in-time recovery enable reversion in seconds, not minutes. Rollback procedures are tested in UAT — never for the first time in production.

Promotion Criteria: DEV → TEST

The Definition of Ready for each stage transition is a formal quality gate — not a suggestion. The following criteria must be fully satisfied before code is promoted from Development to Testing.

Mandatory Criteria

- [x] ~~Unit tests pass — 100% success rate, ≥90% code coverage~~
- [x] ~~No critical SonarQube issues (zero blocker/critical findings)~~
- [x] ~~Integration tests pass locally with Testcontainers~~
- [x] ~~Code review approved by minimum 2 senior reviewers~~
- [x] ~~No hardcoded secrets, credentials, or API keys in code~~
- [x] ~~API documentation updated (OpenAPI / Swagger specification)~~
- [x] ~~Feature flag implemented — can disable in production without redeployment~~

> **Why Feature Flags Matter:** Feature flags allow a deployed feature to be disabled instantly in production without a rollback deployment. For a UPI payment feature, this means a compliance issue discovered post-deployment can be resolved in seconds — not the 30 minutes a full rollback requires.

Automated Gate Enforcement

These criteria are enforced programmatically by the CI/CD pipeline. A failed gate blocks the merge to main branch and notifies the developer immediately — preventing the accumulation of technical debt that degrades environment quality over time.

Promotion Criteria: TEST → UAT and UAT → PRODUCTION

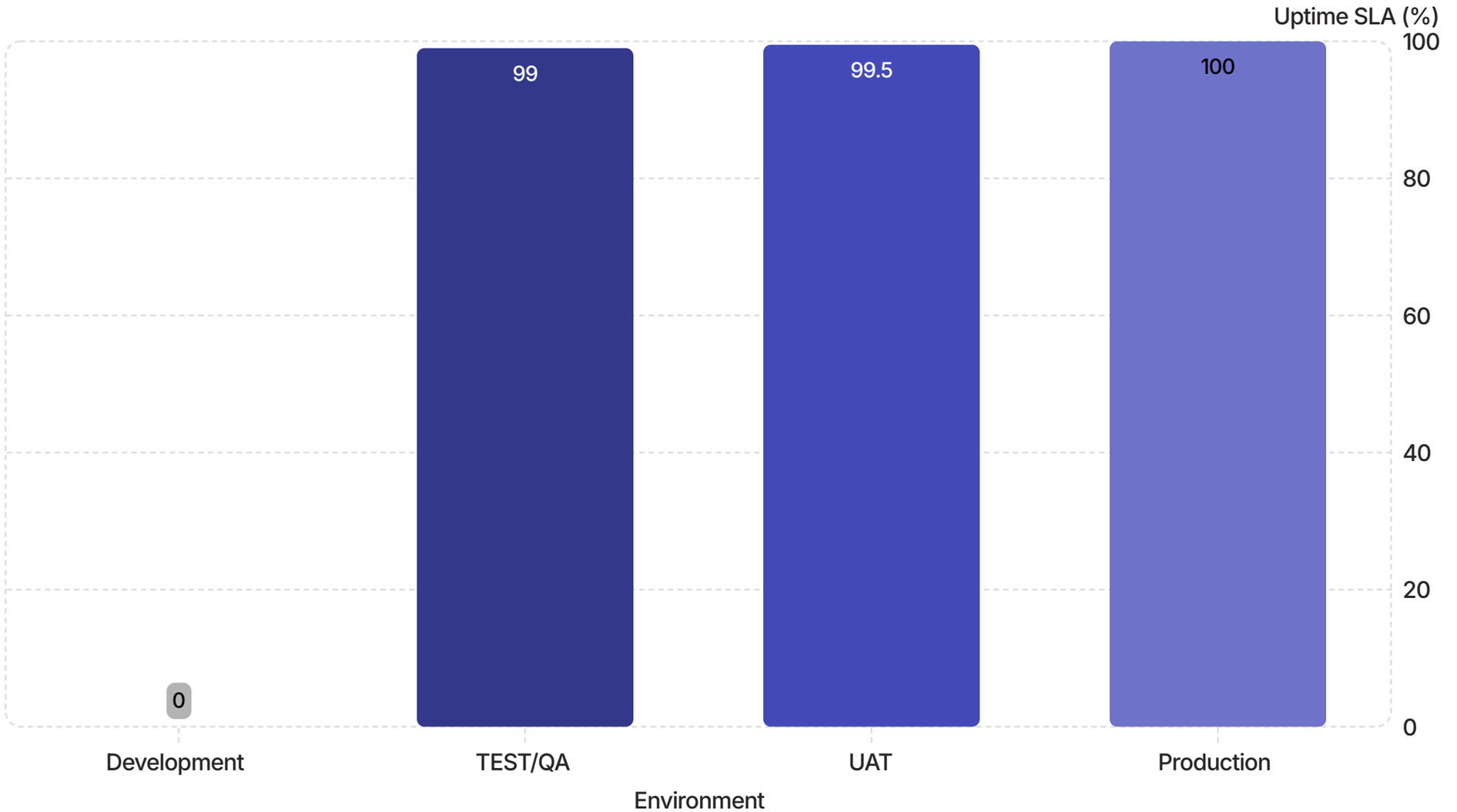
TEST → UAT Promotion Criteria

- [x] ~~All automated tests pass (500+ integration, regression, contract)~~
- [x] ~~Performance tests pass — p99 <200ms at 2× peak load~~
- [x] ~~Security scan passes — no critical CVEs, no SQL injection~~
- [x] ~~Chaos tests pass — Redis failure, DB failure, network latency~~
- [x] ~~Code coverage >80% (integration + unit combined)~~
- [x] ~~No open P1 or P2 defects~~
- [x] ~~Data masking verified — no real PAN in TEST environment~~

UAT → PRODUCTION Promotion Criteria

- [x] ~~Business sign-off — Product Owner formal approval~~
- [x] ~~Compliance sign-off — RBI, PCI-DSS, legal team~~
- [x] ~~Customer validation — beta test success rate >95%~~
- [x] ~~No open P1, P2, or P3 defects~~
- [x] ~~Rollback plan documented and tested in UAT~~
- [x] ~~Runbook updated — on-call team briefed on diagnostics~~
- [x] ~~Monitoring dashboards ready — SLOs and business metrics configured~~
- [x] ~~Canary deployment configured — 1% → 10% → 50% → 100%~~
- [x] ~~Feature flag in place — disable without redeployment~~
- [x] ~~Change Advisory Board (CAB) formal approval obtained~~

Uptime SLA Comparison Across Environments



The progression from zero SLA in Development to 99.99% in Production reflects the increasing business criticality of each environment. The difference between 99.5% (UAT) and 99.99% (Production) represents the difference between 43 hours of allowable downtime per year versus just 52 minutes — a 50× tighter constraint that demands fundamentally different operational practices.

Summary: The Pipeline in One Sentence

> "Dev is for building it right, Testing is for building it correctly, UAT is for building the right thing, and Production is where the money actually moves."

DEV "Does it work?" Engineers, synthetic data, fast iteration, unit tests, no compliance	**TEST** "Does it work correctly?" QA, automated suites, performance, security, 500+ test cases
UAT "Is it the right thing?" Business, compliance, real scenarios, RBI sign-off, beta users	**PROD** "Does it work for real customers?" Live traffic, canary deployment, 24/7 SRE, 99.99% SLA, real money